Software Engineering (14:332:452/567), Fall 2026

Rutgers University

Agentic AI Study Planner and Task Management Assistant

Proposal Report

Team Members

Ketan Singh

Charvi Shashtri

Chaitanya Manik

Chaitya Dobariya

Aayush Patel

Smit Kathiriya

October 9, 2026

Contents

1. Project Proposal 3

2. Requirements 7

3. Use Cases 10

4. Team Contributions 14

1. Project Proposal

1.1 Overview

For our project we plan to build a web app that helps students plan their study time. The student enters their assignments, exams, deadlines and the hours they have free during the week, and the app produces a weekly study schedule. The part we care about most is what happens after that. Schedules rarely survive the week, so when a student finishes something early, skips a session or gets a new assignment, the app should rework the rest of the plan instead of leaving it out of date.

The "agentic" part of the name refers to an LLM-based planning component that breaks bigger goals into smaller tasks, suggests study blocks, and revises them when progress changes. The rest of the app (accounts, data storage, checking deadlines) is regular application code.

1.2 Problem Statement

Most students already have somewhere to keep track of deadlines, whether that is Canvas, Google Calendar, a notes app or a to-do list. The issue we kept coming back to as a group is that none of these actually help you decide what to work on tonight or how long to spend on it. Knowing that a project is due Friday doesn't tell you how to fit it around two other courses.

From our own experience we identified three specific problems:

1. Deadlines are spread out. Each course posts work in a slightly different place, and personal commitments live somewhere else again, so getting a full picture of the week takes manual effort.
2. Plans go out of date quickly. If an assignment planned for one hour on Tuesday ends up taking two, the quiz review and project work after it get pushed back too, and by Thursday the original plan is no longer realistic.
3. Fixing the plan is its own chore. Every time something slips, the student has to decide again what to move and what to cut. That is time spent planning rather than studying.

1.3 Project Goals

Our main goal is to cut down the time students spend planning, while leaving the actual decisions with them. The student sets priorities and deadlines. The app suggests a schedule and explains any changes it makes, and the student can accept, edit or throw out any part of it. We do not want a tool that quietly rearranges someone's week.

For the first version, we consider the project successful if a student can:

- turn a list of deadlines into specific study sessions on specific days,
- report progress and get an updated plan for the rest of the week, and

- change or reject any session the app suggests.

1.4 Technical Approach and Architecture

We split the system into four parts: a React frontend, a Node.js/Express backend, a MongoDB database, and a separate planning agent that calls an LLM. We chose this layout so the AI piece stays one component of the app instead of being mixed into everything, which should also make it easier to test the rest of the code without it.

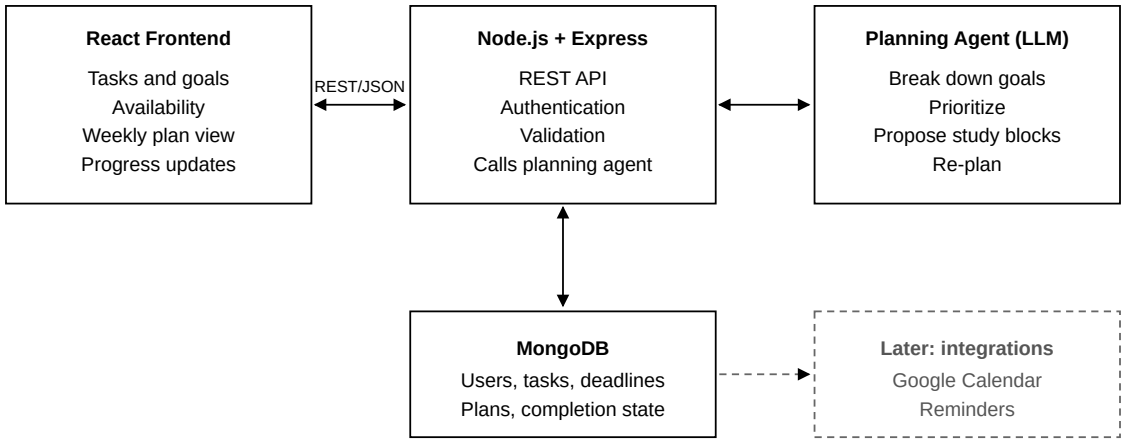


Figure 1. High-level architecture

Frontend (React). This is where the student adds tasks and goals, sets which hours they are free, looks at the weekly plan, edits sessions and marks progress. It talks to the backend over a REST API using JSON.

Backend (Node.js and Express). Handles login, checks incoming data, applies the scheduling rules and decides when to call the planning agent.

Database (MongoDB). Stores user accounts, tasks, deadlines, availability, saved versions of each plan and which sessions have been completed.

Planning agent. When a plan is requested, the backend sends the agent a structured summary of the student's tasks, deadlines and free time. The agent uses an LLM to split large tasks into smaller steps, put them in a sensible order and propose study blocks. It goes through the same steps whenever a re-plan is needed: look at the current tasks, weigh priorities against available time, propose blocks, save the plan once the student accepts it, and adjust again after the next progress update.

One design decision we want to point out is that the LLM's output is never shown to the student directly. The backend first checks it with ordinary code, for example making sure no block lands after its deadline or outside the student's free hours. If something fails those checks, it is fixed or dropped.

This keeps the important rules testable and stops the model from changing deadlines or saved data by mistake. Calendar sync and reminders are left for later and should be possible to add without changing this structure.

1.5 Scope

The first release will cover user accounts, adding and editing tasks, deadlines and goals, entering weekly availability, generating a weekly plan with the agent, editing or rejecting suggested sessions, and re-planning after a task is marked complete, delayed or missed. It will also include unit and acceptance tests.

We are leaving out Google Calendar sync, notifications and reminders, automatic import of deadlines from Canvas, and a native mobile app. These would be useful but are not needed to show the main idea, and we would rather finish a smaller set of features properly.

1.6 Timeline

We plan to build a working planner without AI first and then add the agent on top. That way, if the AI integration runs late, we still have a usable app to show. The dates below are our own targets and may move a bit around the course checkpoints.

Phase	Target	Work planned	Done when
M1: Foundation	Mid October	Set up the GitHub repo and CI, authentication, data model, basic React app	A student can create and save tasks
M2: Core planner	Late October	Task create/edit/delete, availability input, weekly calendar or list view	A student can see a usable weekly plan
M3: AI planning	November	Goal breakdown, schedule generation, re-planning after progress updates	The agent can generate and update a plan
M4: Testing and delivery	Early December	Jasmine unit tests, Cucumber acceptance tests, integration testing, bug fixes, documentation, demo	All acceptance scenarios pass end to end

1.7 Deliverables

By the end of the semester we expect to hand in the working web application with AI planning and re-planning, saved tasks and plans, a suite of Jasmine unit tests and Cucumber acceptance tests, project documentation, and a live demo of the main user flow.

1.8 Risks

Risk	How we plan to handle it
The LLM suggests something invalid, like a session after the deadline	Every suggestion goes through backend validation before it is saved or displayed.
The AI integration takes longer than expected	The non-AI planner is finished by M2, and the agent is behind a single interface so the rest of the app doesn't depend on it.
Students don't trust automatic changes	Changes are shown as suggestions with a short reason, and nothing replaces a student's own edits without them confirming.
The scope grows during development	Calendar sync and reminders are explicitly marked as future work.

2. Requirements

2.1 Users and Actors

The main user is a student managing coursework across several classes. The planning agent is treated as a system actor since it takes part in generating and updating plans. An external calendar service would be a third actor once calendar sync is added, but it is not part of the first release.

2.2 User Stories

We used three user stories to define the minimum version of the product. Since we plan to use Cucumber for acceptance testing, we also wrote a sample scenario for each one.

US-1: Create a realistic study plan

As a student, I want to enter my tasks, deadlines and available time so that I get a study plan I can actually follow.

Acceptance criteria: the plan schedules all work for a task before its deadline and only uses time the student marked as available.

```
Scenario: Plan fits before the deadline
  Given I have a task "ML assignment" due Friday that needs 4 hours
  And I am free Monday to Thursday from 6 PM to 9 PM
  When I ask for a study plan
  Then all sessions for "ML assignment" are before Friday
  And no session is outside my free hours
```

US-2: Update the plan as I make progress

As a student, I want to mark tasks as complete, delayed or missed so that the rest of my schedule adjusts automatically.

Acceptance criteria: marking a task complete or delayed produces an updated plan. Only the affected work is moved; the rest of the week stays the same.

```
Scenario: Missed session gets moved
  Given my plan has "Project slides" on Tuesday at 7:30 PM
  When I mark that session as missed
  Then the planner suggests a later free slot before the deadline
  And sessions for other tasks do not change
```

US-3: Stay in control of the suggestions

As a student, I want to review and edit the suggested schedule so that the planner works around my preferences instead of overriding them.

Acceptance criteria: every generated session can be moved, edited or rejected.

```
Scenario: Moving a suggested session
  Given the planner put "Quiz practice" on Wednesday at 9 PM
  When I move it to Thursday at 6 PM
  Then the plan saves my change
  And later re-plans leave that session where I put it
```

2.3 Functional Requirements

ID	The system shall...	Related	Priority
FR-1	let a student sign up, log in and log out.	All	High
FR-2	let a student add, view, edit and delete tasks with a title, course, deadline, estimated hours and priority.	US-1	High
FR-3	let a student enter larger goals that can be split into smaller tasks.	US-1	Medium
FR-4	let a student set the hours they are free each week.	US-1	High
FR-5	generate a weekly study plan using the planning agent when the student asks for one.	US-1	High
FR-6	check each suggested session against deadlines and availability before showing or saving it.	US-1	High
FR-7	show the plan in a weekly calendar or list view.	US-1	High
FR-8	let a student mark a task or session as complete, delayed or missed.	US-2	High
FR-9	update the remaining schedule after progress changes or a new task is added, changing only the sessions that are affected.	US-2	High
FR-10	give a short reason for each change it suggests.	US-2, US-3	Medium
FR-11	let a student accept, move, edit or reject any suggested session.	US-3	High
FR-12	keep sessions the student edited by hand in place during later re-plans.	US-3	High
FR-13	save tasks, availability, plan versions and completion status.	All	High
FR-14	warn the student when there isn't enough time left to finish a task before its deadline.	US-1, US-2	Medium
FR-15	(future) sync plans with Google Calendar and send reminders.	-	Low

2.4 Non-Functional Requirements

ID	Type	Requirement
NFR-1	Usability	A first-time user should be able to add tasks and get a plan without reading instructions.
NFR-2	Performance	Regular actions like loading or editing tasks should respond in about a second. While the agent is generating a plan, the UI shows that it is working.
NFR-3	Reliability	If the agent fails or returns bad output, the student's current plan and data are left unchanged and an error message is shown.
NFR-4	Security	Passwords are hashed, API routes require login, and students can only see their own data.
NFR-5	Privacy	Only the task and availability information needed for planning is sent to the LLM.
NFR-6	Maintainability	The planning agent sits behind one backend interface so it can be replaced or mocked in tests.
NFR-7	Testability	Core logic has Jasmine unit tests, and each user story has Cucumber scenarios that run in CI.
NFR-8	User control	The AI never changes deadlines, priorities or sessions the student edited unless the student confirms it.

2.5 Constraints and Assumptions

- Tech stack: React, Node.js with Express, MongoDB, an LLM API, Jasmine and Cucumber for testing, GitHub with CI.
- Development runs from mid October to early December 2026.
- For the first release, students type in their own deadlines and time estimates. Nothing is imported from Canvas.
- The app is used through a normal web browser on a laptop or phone.

3. Use Cases

3.1 Use Case Diagram

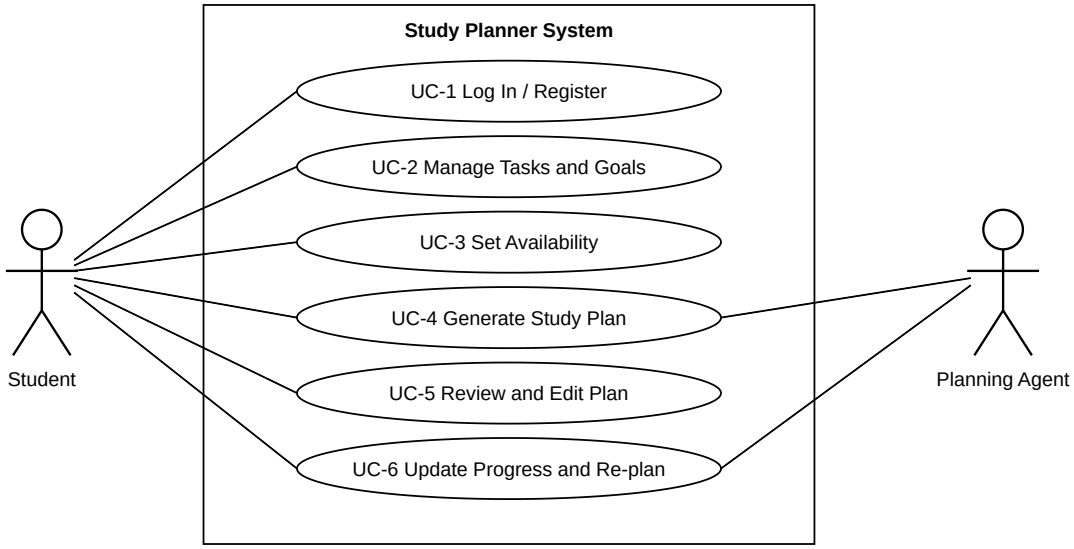


Figure 2. Use case diagram for the first release

3.2 Use Case Descriptions

UC-1: Log In / Register

Actor	Student
Description	The student creates an account or logs in to get to their own planner.
Precondition	None.
Main flow	1. Student opens the app and picks sign up or log in. 2. Student enters email and password. 3. System checks the details and creates the account or verifies the login. 4. System opens the student's dashboard.
Alternate flow	3a. Email is already registered or the password is wrong. The system shows an error and the student tries again.
Postcondition	Student is logged in and can only see their own data.

UC-2: Manage Tasks and Goals

Actor	Student
Description	The student adds, edits or deletes tasks and goals.
Precondition	Student is logged in.
Main flow	1. Student clicks "Add task" (or "Add goal"). 2. Student fills in the title, course, deadline, estimated hours and priority. 3. System checks that required fields are filled and the deadline hasn't passed. 4. System saves the task and shows it in the list.
Alternate flows	2a. Student edits or deletes an existing task. The system saves the change and flags the current plan for re-planning (UC-6). 3a. A field is missing or invalid. The system highlights it and doesn't save.
Postcondition	The task or goal is saved.

UC-3: Set Availability

Actor	Student
Description	The student tells the planner which hours they can study.
Precondition	Student is logged in.
Main flow	1. Student opens the availability page. 2. Student selects free time slots for each day. 3. System checks that the slots don't overlap and saves them.
Alternate flow	3a. Availability changes while a plan already exists. The system asks if the student wants to re-plan (UC-6).
Postcondition	Weekly availability is saved.

UC-4: Generate Study Plan

Actors	Student, Planning Agent
Description	The system turns the student's tasks and free time into study sessions for the week.
Precondition	Student has at least one task and has set availability.
Main flow	1. Student clicks "Plan my week". 2. Backend collects tasks, deadlines, priorities and availability and sends them to the planning agent. 3. Agent breaks large tasks into smaller steps and proposes study sessions. 4. Backend checks each session against deadlines and availability. 5. System shows the suggested plan in the weekly view. 6. Student reviews the plan (UC-5).
Alternate flows	4a. A session breaks a rule. The backend removes or adjusts it. 4b. There isn't enough time to finish a task before its deadline. The system warns the student which task is at risk. 3a. The agent fails or times out. The system shows an error and the existing plan stays as it was.
Postcondition	A checked plan is saved as a new plan version.

UC-5: Review and Edit Plan

Actor	Student
Description	The student goes through the suggested plan and makes changes.
Precondition	A suggested plan exists (from UC-4 or UC-6).
Main flow	1. Student looks over the suggested sessions and the reasons given. 2. Student accepts, moves, edits or rejects individual sessions. 3. System checks each change and saves it. 4. Student accepts the plan and it becomes the active plan.
Alternate flows	3a. A change would put work after the deadline. The system warns the student, who can keep it anyway or pick another time. 2a. Student rejects the whole plan. The previous plan stays active.
Postcondition	The active plan reflects the student's choices, and hand-edited sessions are marked so later re-plans don't move them.

UC-6: Update Progress and Re-plan

Actors	Student, Planning Agent
Description	The student reports progress and the plan is updated to match.
Precondition	The student has an active plan.
Main flow	1. Student marks a session or task as complete, delayed or missed. 2. System saves the new status. 3. System sends the remaining work and updated status to the planning agent. 4. Agent suggests changes only to the affected sessions and leaves hand-edited ones alone. 5. Backend checks the suggestions and shows the student what changed and why. 6. Student accepts or edits the changes (UC-5).
Alternate flows	1a. A task is finished early. The freed-up time is offered to the next most important task. 1b. A new task is added or availability changes. The flow continues from step 3. 4a. The remaining work no longer fits. The system warns the student and suggests what could be shortened or moved.
Postcondition	The updated plan is saved as a new version and progress history is kept.

3.3 Mapping Stories to Use Cases and Requirements

User story	Use cases	Requirements	Milestone
US-1	UC-2, UC-3, UC-4	FR-2 to FR-7, FR-14	M2 (without AI), M3
US-2	UC-6	FR-8, FR-9, FR-10	M3
US-3	UC-5	FR-10, FR-11, FR-12	M2, M3
All	UC-1	FR-1, FR-13	M1

4. Team Contributions

Member	Work on the proposal	Contribution
Ketan Singh	Title and introduction, overall project framing	16.7%
Charvi Shashtri	Problem statement and project goals	16.7%
Chaitanya Manik	User stories and acceptance criteria	16.7%
Chaitya Dobariya	Technical approach and architecture	16.7%
Aayush Patel	Timeline and milestones	16.6%
Smit Kathiriya	Closing and deliverables	16.6%
	Total	**100%**

All six members also worked on the requirements and use cases in this report and rehearsed the presentation together.